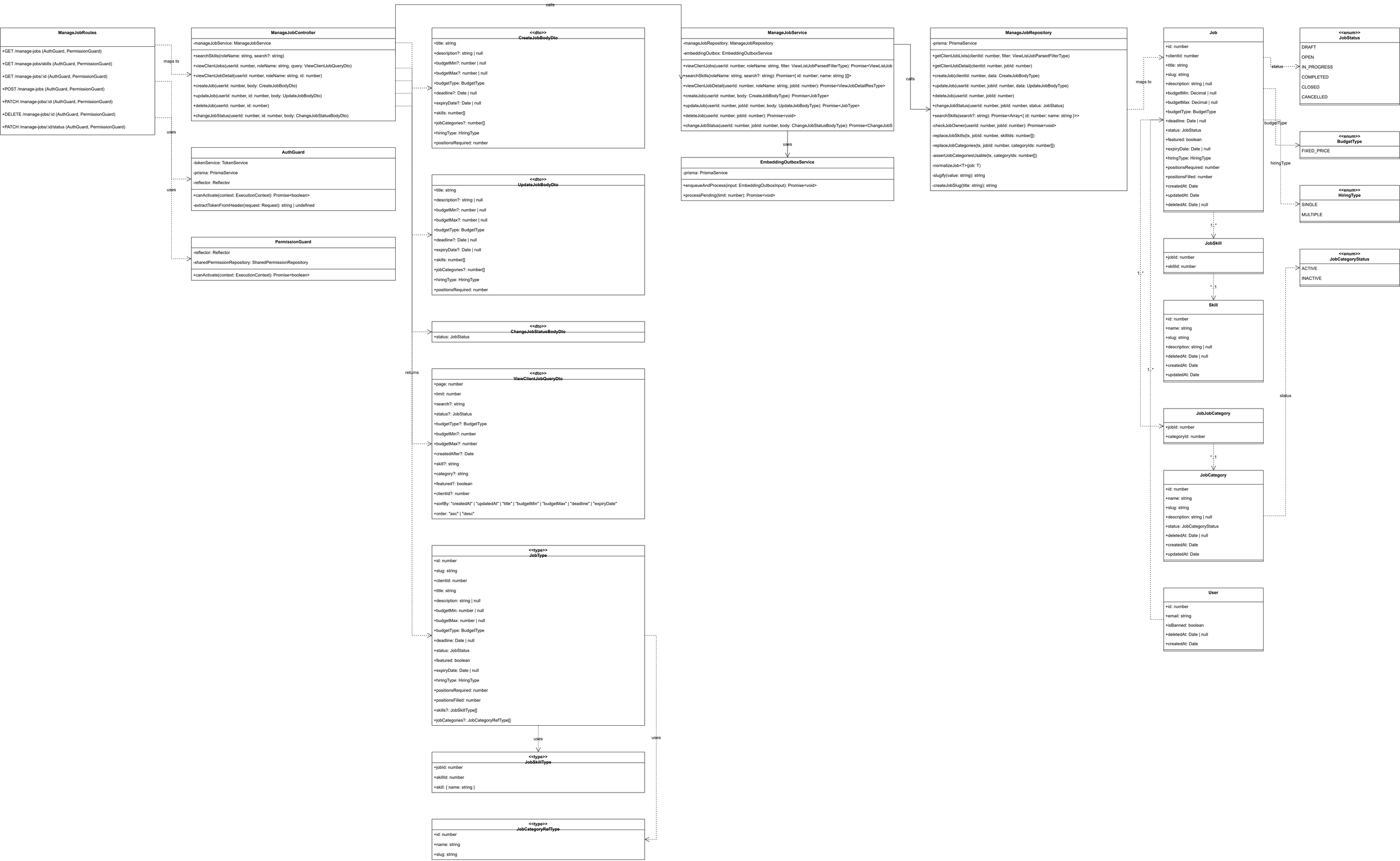

ManageJobRoutes
+GET /manage-jobs (AuthGuard, PermissionGuard)
+GET /manage-jobs/skills (AuthGuard, PermissionGuard)
+GET /manage-jobs/:id (AuthGuard, PermissionGuard)
+POST /manage-jobs (AuthGuard, PermissionGuard)
+PATCH /manage-jobs/:id (AuthGuard, PermissionGuard)
+DELETE /manage-jobs/:id (AuthGuard, PermissionGuard)
+PATCH /manage-jobs/:id/status (AuthGuard, PermissionGuard)
maps to
uses
uses
ManageJobController
-manageJobService: ManageJobService
+searchSkills(roleName: string, search?: string)
+viewClientJobs(userId: number, roleName: string, query: ViewClientJobQueryDto)
+viewClientJobDetail(userId: number, roleName: string, id: number)
+createJob(userId: number, body: CreateJobBodyDto)
+updateJob(userId: number, id: number, body: UpdateJobBodyDto)
+deleteJob(userId: number, id: number)
+changeJobStatus(userId: number, id: number, body: ChangeJobStatusBodyDto)
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boolean>
-extractTokenFromHeader(request: Request): string | undefined
PermissionGuard
-reflector: Reflector
-sharedPermissionRepository: SharedPermissionRepository
+canActivate(context: ExecutionContext): Promise<boolean>
calls
<<dto>>
CreateJobBodyDto
+title: string
+description?: string | null
+budgetMin?: number | null
+budgetMax?: number | null
+budgetType: BudgetType
+deadline?: Date | null
+expiryDate?: Date | null
+skills: number[]
+jobCategories?: number[]
+hiringType: HiringType
+positionsRequired: number
<<dto>>
UpdateJobBodyDto
+title: string
+description?: string | null
+budgetMin?: number | null
+budgetMax?: number | null
+budgetType: BudgetType
+deadline?: Date | null
+expiryDate?: Date | null
+skills: number[]
+jobCategories?: number[]
+hiringType: HiringType
+positionsRequired: number
<<dto>>
ChangeJobStatusBodyDto
+status: JobStatus
returns
<<dto>>
ViewClientJobQueryDto
+page: number
+limit: number
+search?: string
+status?: JobStatus
+budgetType?: BudgetType
+budgetMin?: number
+budgetMax?: number
+createdAfter?: Date
+skill?: string
+category?: string
+featured?: boolean
+clientId?: number
+sortBy: "createdAt" | "updatedAt" | "title" | "budgetMin" | "budgetMax" | "deadline" | "expiryDate"
+order: "asc" | "desc"
<<type>>
JobType
+id: number
+slug: string
+clientId: number
+title: string
+description: string | null
+budgetMin: number | null
+budgetMax: number | null
+budgetType: BudgetType
+deadline: Date | null
+status: JobStatus
+featured: boolean
+expiryDate: Date | null
+hiringType: HiringType
+positionsRequired: number
+positionsFilled: number
+skills?: JobSkillType[]
+jobCategories?: JobCategoryRefType[]
uses
uses
<<type>>
JobSkillType
+jobId: number
+skillId: number
+skill: { name: string }
<<type>>
JobCategoryRefType
+id: number
+name: string
+slug: string
ManageJobService
-manageJobRepository: ManageJobRepository
-embeddingOutbox: EmbeddingOutboxService
+viewClientJobs(userId: number, roleName: string, filter: ViewListJobParsedFilterType): Promise<ViewListJob
+searchSkills(roleName: string, search?: string): Promise<{ id: number; name: string }[]>
+viewClientJobDetail(userId: number, roleName: string, jobId: number): Promise<ViewJobDetailResType>
+createJob(userId: number, body: CreateJobBodyType): Promise<JobType>
+updateJob(userId: number, jobId: number, body: UpdateJobBodyType): Promise<JobType>
+deleteJob(userId: number, jobId: number): Promise<void>
+changeJobStatus(userId: number, jobId: number, body: ChangeJobStatusBodyType): Promise<ChangeJobS
uses
EmbeddingOutboxService
-prisma: PrismaService
+enqueueAndProcess(input: EmbeddingOutboxInput): Promise<void>
+processPending(limit: number): Promise<void>
calls
ManageJobRepository
-prisma: PrismaService
+getClientJobLists(clientId: number, filter: ViewListJobParsedFilterType)
+getClientJobDetail(clientId: number, jobId: number)
+createJob(clientId: number, data: CreateJobBodyType)
+updateJob(userId: number, jobId: number, data: UpdateJobBodyType)
+deleteJob(userId: number, jobId: number)
+changeJobStatus(userId: number, jobId: number, status: JobStatus)
+searchSkills(search?: string): Promise<Array<{ id: number; name: string }>>
-checkJobOwner(userId: number, jobId: number): Promise<void>
-replaceJobSkills(tx, jobId: number, skillIds: number[])
-replaceJobCategories(tx, jobId: number, categoryIds: number[])
-assertJobCategoriesUsable(tx, categoryIds: number[])
-normalizeJob<T>(job: T)
-slugify(value: string): string
-createJobSlug(title: string): string
maps to
Job
+id: number
+clientId: number
+title: string
+slug: string
+description: string | null
+budgetMin: Decimal | null
+budgetMax: Decimal | null
+budgetType: BudgetType
+deadline: Date | null
+status: JobStatus
+featured: boolean
+expiryDate: Date | null
+hiringType: HiringType
+positionsRequired: number
+positionsFilled: number
+createdAt: Date
+updatedAt: Date
+deletedAt: Date | null
1..*
JobSkill
+jobId: number
+skillId: number
*..1
Skill
+id: number
+name: string
+slug: string
+description: string | null
+deletedAt: Date | null
+createdAt: Date
+updatedAt: Date
1..*
JobJobCategory
+jobId: number
+categoryId: number
*..1
JobCategory
+id: number
+name: string
+slug: string
+description: string | null
+status: JobCategoryStatus
+deletedAt: Date | null
+createdAt: Date
+updatedAt: Date
1..*
User
+id: number
+email: string
+isBanned: boolean
+deletedAt: Date | null
+createdAt: Date
status
budgetType
hiringType
status
<<enum>>
JobStatus
DRAFT
OPEN
IN_PROGRESS
COMPLETED
CLOSED
CANCELLED
<<enum>>
BudgetType
FIXED_PRICE
<<enum>>
HiringType
SINGLE
MULTIPLE
<<enum>>
JobCategoryStatus
ACTIVE
INACTIVE